

Concurrency II – COMPLETE THEORY & PRACTICAL NOTES

Quote

"Concurrency is one of the most complex problems in computer science because bugs are rare, non-deterministic, and extremely hard to reproduce."

1. Client/Server Model (Theory)

A server waits for incoming connections, processes requests, and sends responses. In a non-threaded model, requests are processed sequentially, causing bottlenecks.

```
ServerSocket serverSocket = new ServerSocket(8009);
while (true) {
    Socket socket = serverSocket.accept();
    process(socket);
}
```

Problem Analysis

This model is blocking. If one request is slow (I/O or computation), all other clients must wait.

2. Throughput Theory

Throughput = number of tasks completed per unit time. There are two bottlenecks: - I/O Bound → waiting (network, disk) - CPU Bound → computation heavy Threads improve performance only for I/O-bound systems.

3. Multithreading Improvement

Threads allow overlapping work. While one thread waits (I/O), another runs.

```
void process(Socket socket) {
    new Thread(() -> {
        handle(socket);
    }).start();
}
```

Critical Problem

Unlimited threads → memory exhaustion, context switching overhead.

4. Clean Concurrency Design

Concurrency must follow Single Responsibility Principle: - Connection handling - Business logic - Thread management Thread logic must be isolated.

5. Thread Scheduler Abstraction

Separate thread policy from logic.

```
public interface ClientScheduler {
    void schedule(Runnable task);
}

public class ThreadPerRequestScheduler implements ClientScheduler {
    public void schedule(Runnable task) {
        new Thread(task).start();
    }
}
```

6. Executor Framework (Best Practice)

Java provides thread pools to control concurrency.

```
ExecutorService executor = Executors.newFixedThreadPool(10);

executor.execute(() -> {
    processRequest();
});
```

Advantages

- Limits thread creation - Reuses threads - Better performance - Easier maintenance

7. Race Condition (Theory)

A race condition occurs when multiple threads access shared data and at least one modifies it.

```
int value = 0;

Thread 1: value++;
Thread 2: value++;
```

Explanation

Increment is NOT atomic: 1. Read 2. Add 3. Write Threads can interleave → lost update.

8. Atomic Operations

Atomic = cannot be interrupted. Example: simple assignment is atomic, but ++ is NOT.

9. Bytecode Reality

++ becomes multiple JVM instructions → can interleave between threads → causes bugs.

10. Synchronization

Use synchronized to ensure only one thread enters critical section.

```
public synchronized int increment() {  
    return ++value;  
}
```

Tradeoff

Safe but slower due to locking.

11. Non-blocking (Atomic Classes)

Use CAS (Compare-And-Swap).

```
AtomicInteger value = new AtomicInteger(0);  
value.incrementAndGet();
```

Concept

If value unchanged → update Else → retry

12. Method Dependency Problem

Even synchronized methods can fail if used together.

```
if(iterator.hasNext()) {  
    iterator.next();  
}
```

Issue

Another thread may change state between calls → inconsistent behavior.

13. Locking Strategies

Client-side locking: caller handles synchronization (error-prone) Server-side locking: object manages synchronization (recommended)

14. Deadlock

Deadlock = threads wait forever.

4 Conditions

1. Mutual Exclusion 2. Hold & Wait 3. No Preemption 4. Circular Wait

```
Thread1: lock(A) → wait B
Thread2: lock(B) → wait A
```

Solution

Enforce global lock ordering.

16. Deadlock Prevention Strategies

- Avoid circular wait - Use timeout - Use ordering - Use atomic/non-blocking

17. Throughput Calculation

Single thread: Total time = (I/O + CPU) * N Multi-thread: Overlap I/O and CPU → higher throughput

18. Testing Concurrency

Concurrency bugs are rare → need repeated execution.

```
for(int i=0; i<100000; i++){
    runTest();
}
```